HW 2

Sam Ly

February 9, 2026

Chapter 4

1. The lexical grammars of Python and Haskell are not regular. What does that mean, and why aren't they?

   The lexical grammars of Python and Haskell are not regular because the levels of indentation are significant. Regular languages can not have "n-length string of characters" have significant semantic meaning.

2. Aside from separating tokens—distinguishing print foo from printfoo—spaces aren't used for much in most languages. However, in a couple of dark corners, a space does affect how code is parsed in CoffeeScript, Ruby, and the C preprocessor. Where and what effect does it have in each of those languages?

   In C, the macro system places significance on the spaces, because C macros can either be a regular value (eg. number literal, string, etc.) or a "function" that gets subsituted.

   ```
   #define FOO(x) 2*x
   #define BAR (x)
   // Usage:
   FOO(a); // changes to 2*a
   BAR; // changes to (x), likely an error.
   ```

3. Our scanner here, like most, discards comments and whitespace since those aren't needed by the parser. Why might you want to write a scanner that does not discard those? What would it be useful for?

   Many modern languages can actually make use of comments. For example, JavaDoc can turn comments into documentation. The JavaDoc can also inform the IDE's LSP, allowing for inline completions and context.

4. Add support to Lox's scanner for C-style /* ... */ block comments. Make sure to handle newlines in them. Consider allowing them to nest. Is adding support for nesting more work than you expected? Why?

   Adding support for nesting was not very difficult because it is very explicit where the comments start and end. We can recursively call a function that consumes the comment within the block comment method. That way, we can consume all comments properly.

   ```
   // In the scanToken method.
   case '/':
       if (match('/')) {
           // A comment goes until the end of the line.
           while (peek() != '\n' && !isAtEnd()) advance();
       } else if (match('*')) {
           blockComment();
       } else {
           addToken(SLASH);
   ```

```
                }
                break;

    ...
    private void blockComment() {
        while (!isAtEnd()) {
            char curr = advance();
            if (curr == '\n') line++;
            if (curr == '*' && match('/')) {
                break;
            }
            if (curr == '/' && match('*')) {
                blockComment();
            }
        }
    }
```

Chapter 5

1. I would write this as:

```
field -> "." IDENTIFIER;
field -> "." IDENTIFIER field;

params -> "";
params -> expr
params -> expr n_params

n_params -> "," expr n_params
n_params -> "," expr

method -> "(" params ")"

expr -> expr field;
expr -> expr method;
expr -> IDENTIFIER;
expr -> NUMBER;
```

   Bonus: This grammar seems to encode (as the hints imply) some kind of object member access. First, the '`.`' notation lets you access members via a identifier, whether that be a field name or method name. Then, it allows you to call the member, assuming that it is a method, using (...params).

   One strange thing is that I notice you can construct something allong the lines of `123(n)` and it would be valid, treating the number as a method name.

2. I chose to use Haskell for this task. For the grammar, I chose Markdown's *inline* syntax. this includes things like **bold**, *italics*, and links, but not "block" styling like lists or headings. Interestingly, Haskell's type system feels like a one-to-one mapping of the grammar! Similarly, we can pattern match each of the expression types and we have a working Markdown AST in Haskell!

```
import Data.Char (isDigit)
import GHC.Natural (Natural)
```

```haskell
data Inline
  = Plain String
  | Italic String
  | Bold String
  | ItalicBold String
  | Code String
  | Link String String
  | ItalicLink String String
  | BoldLink String String
  | ItalicBoldLink String String
  | Image String String
  | LineBreak

instance Show Inline where
  show :: Inline -> String
  show (Plain text) = text
  show (Italic text) = "(*" ++ text ++ "*)"
  show (Bold text) = "(**" ++ text ++ "**)"
  show (ItalicBold text) = "(***" ++ text ++ "***)"
  show (Code text) = "`" ++ text ++ "`"
  show (Link title ref) = "[" ++ title ++ "](" ++ ref ++ ")"
  show (ItalicLink title ref) = "*[" ++ title ++ "](" ++ ref ++ ")*"
  show (BoldLink title ref) = "**[" ++ title ++ "](" ++ ref ++ ")**"
  show (ItalicBoldLink title ref) = "***[" ++ title ++ "](" ++ ref ++ ")***"
  show (Image alt src) = "![" ++ alt ++ "](" ++ src ++ ")"
  show LineBreak = "\n"

renderInlines :: [Inline] -> String
renderInlines = concatMap show

main :: IO ()
main = putStrLn (renderInlines [Plain "Hello ", Bold "World", LineBreak, Code "Done"])
```

3. Done! I implemented it by using `String.join(...)` instead of the `parenthesize` method.